TAREA LAB-0.1

Introducción del Factor 1

En este apartado se analiza cómo influye la potencia del hardware en el rendimiento de un programa. Para ello, se ha ejecutado exactamente el mismo código fuente (PythonA1.py) en dos ordenadores distintos (PC 1 y PC 2) sometiéndolos a las mismas cargas de trabajo (mismos valores de n).

El objetivo es comparar los tiempos de ejecución resultantes y demostrar cómo las características técnicas de cada equipo (principalmente la CPU) afectan a la velocidad de resolución del problema.

Especificaciones de los equipos:

Ordenador 1 (PC_1 - Referencia):

- **Procesador:** 12th Gen Intel(R) Core(TM) i5-12400 @ 2.50 GHz
- **Memoria RAM:** 16,0 GB

Ordenador 2 (PC_2 - Alto Rendimiento):

- **Procesador:** AMD Ryzen 7 7800X3D 8-Core Processor @ 4.20 GHz
- **Memoria RAM:** 32,0 GB

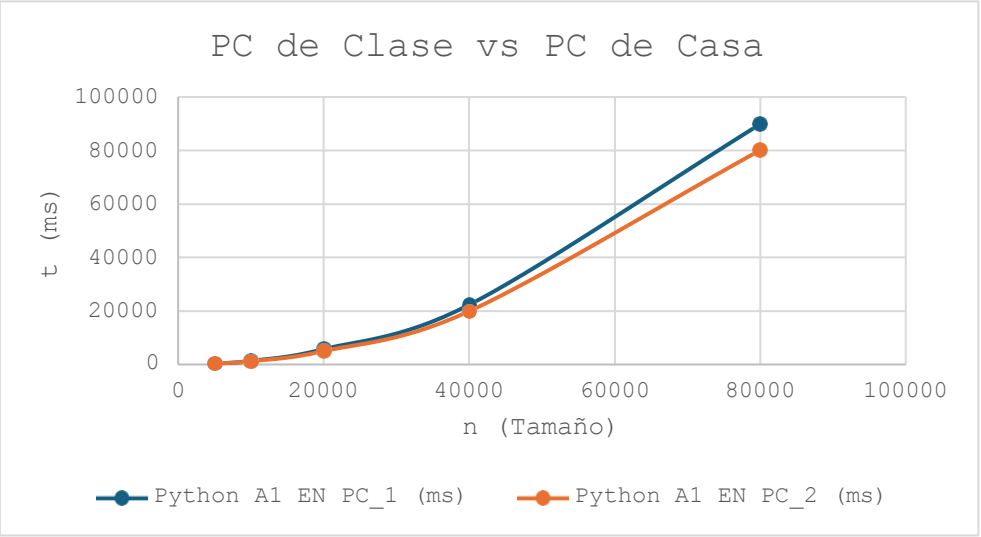
A priori, se espera que el PC_2 obtenga mejores tiempos debido a su mayor frecuencia base (4.20 GHz frente a 2.50 GHz) y a que se trata de un procesador de gama entusiasta frente a la gama media del i5.

Resultados Experimentales

A continuación, se presentan los tiempos obtenidos en milisegundos para ambos equipos:

n (Tamaño)	Python A1 EN PC_1 (ms)	Python A1 EN PC_2 (ms)
5000	335	334
10000	1347	1242
20000	5738	4999
40000	22318	19928
80000	89896	80168

A continuación, se adjunta también una gráfica comparativa:



Conclusión del Factor 1

Se demuestra que un hardware más potente (mayor frecuencia de CPU y mejor arquitectura) reduce el tiempo de ejecución de manera constante. Sin embargo, dado que el Algoritmo A1 es ineficiente por naturaleza (complejidad cuadrática).

Introducción del Factor 2

En este apartado se aborda el factor más determinante en el rendimiento: el diseño del algoritmo. Mientras que el hardware o el lenguaje pueden mejorar el tiempo en un factor constante (x2, x10), un buen algoritmo puede reducir la complejidad teórica del problema, logrando mejoras de varios órdenes de magnitud.

Se han implementado y medido tres evoluciones sobre el algoritmo base:

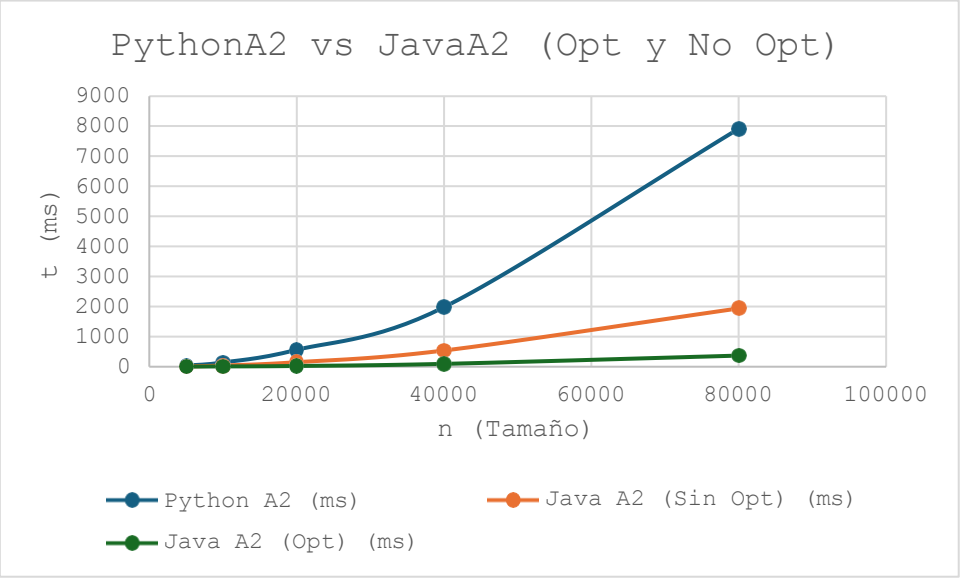
- **Algoritmo A2 (Rotura de bucle):** Mejora respecto a A1 deteniendo la búsqueda en cuanto se encuentra el primer divisor.
- **Algoritmo A3 (Raíz Cuadrada):** Optimización matemática que reduce el espacio de búsqueda. Solo se comprueban divisores hasta $\sqrt{n}$ y se omiten los números pares.
- **Algoritmo A4 (Criba de Eratóstenes):** Cambio de paradigma. En lugar de divisiones por tentativa, utiliza un array de booleanos para "tachar" múltiplos. Cambia tiempo de CPU por consumo de memoria.

Resultados Experimentales

La siguiente tabla muestra los tiempos obtenidos para diferentes tamaños de n tanto en Python como en Java (con y sin optimización JIT):

n (Tamaño)	Python A2 (ms)	Java A2 (Sin Opt) (ms)	Java A2 (Opt) (ms)
5000	37	10	3
10000	141	40	10
20000	561	150	25
40000	1983	540	95
80000	7920	1940	370

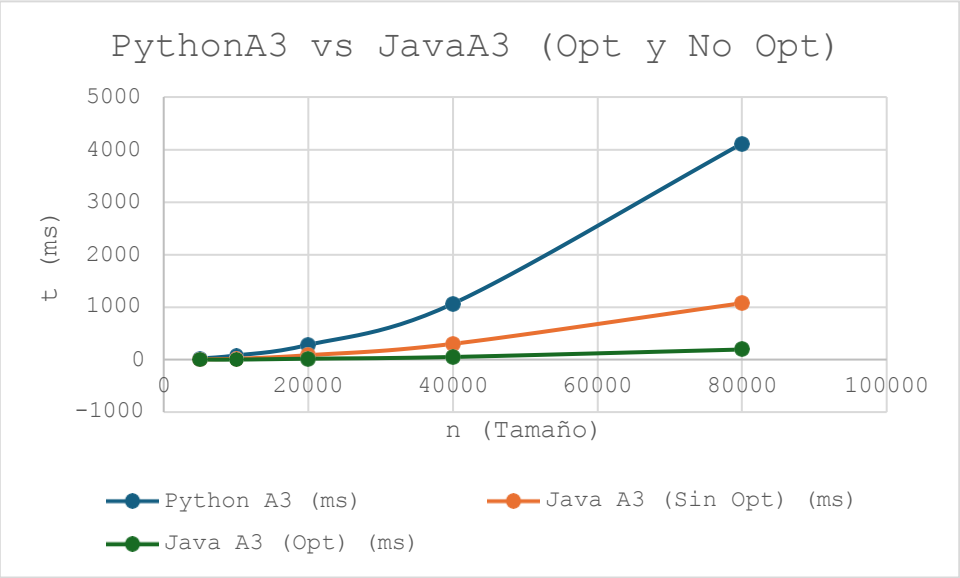
Con su respectiva gráfica:



Ahora el algoritmo A3:

Python A3 (ms)	Java A3 (Sin Opt) (ms)	Java A3 (Opt) (ms)
20	6	0
79	16	0
280	84	17
1064	300	50
4115	1075	192

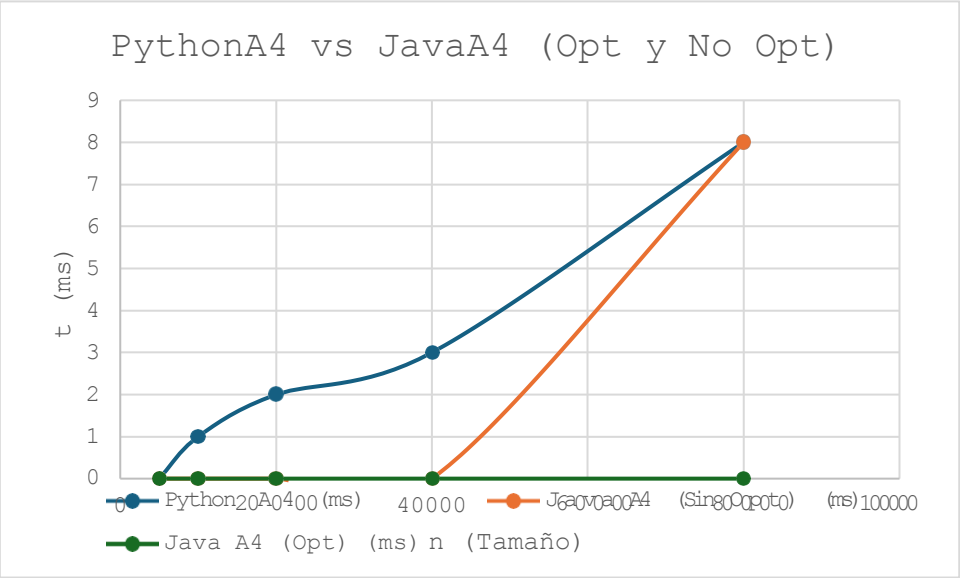
Con su gráfica:



Y, por último, el algoritmo A4 y el más eficiente:

Python A4 (ms)	Java A4 (Sin Opt) (ms)	Java A4 (Opt) (ms)
0	0	0
1	0	0
2	0	0
3	0	0
8	8	0

Con su gráfica:



Diferencia entre Java y Python

Los resultados demuestran claramente que Java es mucho más rápido que Python para este tipo de cálculos. La razón es sencilla:

- **Python (Interpretado):** Funciona como un traductor simultáneo. Lee el código línea por línea y lo traduce al ordenador mientras se ejecuta. Esto hace que pierda tiempo en cada vuelta del bucle "pensando" qué tiene que hacer.
- **Java (Compilado/JIT):** Funciona como si leyera un libro ya traducido. Gracias a su compilador (JIT), transforma el código a un lenguaje que el procesador entiende directamente. Una vez que "aprende" lo que hace el bucle, lo ejecuta a toda velocidad sin detenerse a traducir.

Conclusión

Este experimento valida la premisa fundamental de la asignatura: **El diseño del algoritmo es el factor dominante.**

Mientras que cambiar del PC 1 al PC 2 (Factor 2) mejoraba el tiempo en un 10%, cambiar del Algoritmo A1 al Algoritmo A4 reduce el tiempo en un factor de 10.000 (de casi 90 segundos a 8 milisegundos en Python). Ninguna mejora de hardware puede competir con una reducción en la complejidad algorítmica.

TAREA LAB-1.1

Sesión 02 – P11

Pregunta 1: Calcula, ¿cuántos años más podremos seguir utilizando esta forma de contar? Explica el razonamiento seguido para realizar el cálculo:

El método que se utiliza devuelve un número de tipo long, por tanto, este es un entero de 64 bits. El valor más grande que almacena un long $2^{63} – 1$ lo cual equivale a 9.223.372.036.854.775.807 milisegundos.
Teniendo en cuenta que partimos desde 1970 aproximadamente le quedan 292 millones de años. En ese momento, el sistema colapsará.

Pregunta 2: ¿Por qué a veces el tiempo medido sale 0? ¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?:

Esto se debe a que la medición se hace en ms y los procesadores actuales pueden realizar cuentas como la suma muy rápido, en el orden incluso de los nanosegundos. Además, estamos trabajando con enteros long, por lo que si un algoritmo tarda 0,0005 ms en ejecutarse Java lo redondeará a 0.

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041>java -Xint p11.Vector2 10000000
t1=1770759563056  *** t2=1770759563144
n= 10000000     Tiempo metodo suma = 88
Resultado de la suma de elementos = -292913
```

Según lo medido aproximadamente en *n* = 10000000 el valor ya supera los 50 ms, en concreto, 88 ms.

Mediciones de Vector4.java:

Con parámetro 1:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041>java -Xint p11.Vector4 1
repeticiones = 1
n       Tiempo
10000   0
20000   0
40000   0
80000   0
160000  0
320000  0
640000  7
1280000 10
2560000 22
5120000 33
10240000        55
20480000        106
40960000        201
81920000        411

Fin de la medicion de tiempos *****
```

Con parámetro 10:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041>java -Xint p11.Vector4 10
repeticiones = 10
n       Tiempo
10000   0
20000   0
40000   0
80000   3
160000  14
320000  16
640000  55
1280000 104
2560000 126
5120000 360
10240000        566
20480000        990
40960000        1959
81920000        3975

Fin de la medicion de tiempos *****
```

Con parámetro 100:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041>java -Xint p11.Vector4 100
repeticiones = 100
n       Tiempo
10000   7
20000   10
40000   28
80000   62
160000  116
320000  176
640000  346
1280000 650
2560000 1332
5120000 2550
10240000        4920
20480000        10245
40960000        19548
81920000        38588

Fin de la medicion de tiempos *****
```

Hasta aquí el tiempo era razonable, a partir de aquí la medición se dispara, como podemos ver con 1000 o 10000.

Con parámetro 1000:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041>java -Xint p11.Vector4 1000
repeticiones = 1000
n       Tiempo
10000   91
20000   168
40000   343
80000   471
160000  1306
320000  1992
640000  4716
1280000 7430
2560000 15230
5120000 29434
10240000        58255
20480000        115146
40960000        221243
81920000        415823

Fin de la medicion de tiempos *****
```

Y por último 10000, en la que el tiempo alcanza la hora y media en el último paso:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041>java -Xint p11.Vector4 10000
repeticiones = 10000
n       Tiempo
10000   576
20000   1133
40000   2180
80000   4366
160000  8588
320000  17202
640000  34870
1280000 71006
2560000 140100
5120000 281891
10240000        568375
20480000        1105234
40960000        2217148
81920000        4564232

Fin de la medicion de tiempos *****
```

<u>Pregunta 3:</u> ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?:

Depende directamente de la complejidad del algoritmo, por ejemplo, en Tsuma, Tmaximo y Tcoincidencias2 (O(n)) el tiempo se duplica. Por otro lado, en Tcoincidencias1 ($O(n^2)$) el tiempo se cuadriplica.

<u>Pregunta 4:</u> ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para k=3 y k=4 y compruebe los tiempos obtenidos.)

Pues de manera similar al caso anterior, en caso de O(n) el tiempo sería 3 (k = 3) o 4 veces mayor (k = 4) y en el caso de $O(n^2)$ el tiempo seria inclusive 9 o 16 veces mayor.

<u>Pregunta 5:</u> ¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal O(n)?

En un algoritmo de complejidad O(n), el tiempo de ejecución sigue la función T(n)= c·n (donde c es una constante que depende de la velocidad del procesador y del flag.

Tras programar Vector5.java, Vector6.java y Vector7.java hemos obtenido las siguientes gráficas.
Primero con la comparación de **Tsuma(250) y Tmaximo(250):**

n	*Tsuma*	*Tmaximo*
10000	16	21
20000	39	37
40000	60	74
80000	147	150
160000	250	270
320000	495	533
640000	892	1069
1280000	1850	2156
2560000	3830	4299
5120000	7347	8740
10240000	14574	17289
20480000	28653	34558
40960000	56621	69259
81920000	103941	141083

En este caso podemos ver que los tiempos con el parámetro 250 son bastante similares, ya que ambas son O(n) al ser la Tmaximo más cara computacionalmente que Tsuma se debe la diferencia leve de tiempo, pero no es nada sustancial. En cambio, se espera una diferencia mayor en $O(n^2)$.
A continuación, vemos la diferencia **Tcomparacion1(1) y Tcomparacion2(1):**

n (fdT > 5min)	*Tcoincidencias1*	*Tcoincidencias2*
10000	584	0
20000	2222	0
40000	9114	0
80000	35325	0
160000	140751	0
320000	561340	0
640000	fdT	15
1280000	fdT	16
2560000	fdT	32
5120000	fdT	46
10240000	fdT	95
20480000	fdT	189
40960000	fdT	331
81920000	fdT	679

En el caso anterior vemos que la diferencia es mucho mayor debido a la diferencia de complejidad del algoritmo ya que Tcoindicencias2 es O(n) y Tcoincidencias1 es $O(n^2)$. Para dar un poco más de margen se establece de umbral 5 minutos, pero vemos que ni siquiera llega a la mitad y ya sobrepasa este umbral con solo 1 de parámetro, de hecho, si se hacen cálculos se ve que tardaría en acabar el último *n* más de 500 años.

Indique las características principales (procesador, memoria) del ordenador donde se han medido tiempos.

Características de mi PC:

- Procesador: AMD Ryzen 7 7800X3D 8-Core Processor @ 4.20 GHz
- Memoria RAM: 32,0 GB

TAREA LAB-1.2

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	*TBucle1*	*TBucle2*	*TBucle3*	*TBucle4*
100	0	0	1	0
200	0	0	4	12
400	0	3	15	45
800	0	16	65	379
1600	0	51	267	2803
3200	0	229	1117	22497
6400	1	933	4723	179185
12800	1	4154	20201	Fdt
25600	3	18922	82773	Fdt
51200	7	73948	348930	Fdt

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los cuatro casos.

SE LE PIDE: Implementar tres nuevas clases Bucle5, Bucle6 y Bucle7, que simulen algoritmos iterativos con una complejidad $O(n^2 \log n^2)$, $O(n^3 \log n)$ y $O(n^4)$ respectivamente. Tras implementar las clases, mida sus tiempos de ejecución y rellene la tabla:

Hacemos que partan desde 5:

n	*TBucle5*	*TBucle6*	*TBucle7*
5	1	0	0
10	0	0	0
20	1	0	0
40	0	1	0
80	0	4	12
160	3	25	134
320	16	205	1867

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los tres casos.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	*TBucle1*	*TBucle2*	*t1/t2*
100	0	0	0
200	0	0	0
400	0	3	0
800	0	16	0
1600	0	51	0
3200	0	229	0
6400	1	933	0,001071811
12800	1	4154	0,000240732
25600	3	18922	0,000158546
51200	7	73948	9,46611E-05

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	*TBucle3*	*TBucle2*	*t3/t2*
100	1	0	0
200	4	0	0
400	15	3	5
800	65	16	4,0625
1600	267	51	5,235294118
3200	1117	229	4,877729258
6400	4723	933	5,062165059
12800	20201	4154	4,863023592
25600	82773	18922	4,374431878
51200	348930	73948	4,718586033

CASO3: MISMO ALGORITMO EN ENTORNOS DE DESARROLLO y/o EJECUCIÓN DIFERENTES

Para finalizar, vamos a implementar el mismo algoritmo en dos entornos diferentes con el objetivo de comparar, para ese algoritmo, dichos entornos (Bucle4.py y Bucle4.java). Posteriormente, mediremos los tiempos en Python y Java SIN OPTIMIZACIÓN. El cociente de tiempos para los mismos tamaños del problema oscilará alrededor de un valor (constante que relaciona ambos entornos). Si esta constante es menor a 1, es tanto mejor el que hemos puesto en el numerador y si es mayor que 1, es tanto mejor el que hemos puesto en el denominador

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	*Bucle4.py*	*Bucle4.java*	*Tpy/Tjava*
100	3	0	0
200	22	12	1,833333333
400	177	45	3,933333333
800	1473	379	3,886543536
1600	12601	2803	4,495540492
3200	133328	22497	5,926479086
6400	1136672	179185	6,343566705

TAREA LAB-2.1

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de BubbleSort:

n	*t ordenado*	*t inverso*	*t aleatorio*
10000	423	1096	889
20000	1680	4374	3430
40000	6675	17418	13867
80000	26847	70587	55167
160000	107794	279205	220076

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.
Sí los datos obtenidos concuerdan con lo esperado ya que la complejidad del algoritmo es $O(n^2)$, al duplicar el tamaño del vector de entrada (2n) el número de operación debe multiplicarse por $2^2 = 4$. Por tanto, cuadra ya que siempre el valor siguiente esta en torno a 4 veces el anterior.

En cuanto a los diferentes tiempos para un mismo valor de, también concuerdan con lo esperado. Aunque el algoritmo realiza las mismas comparaciones, el número de intercambios en memoria varía drásticamente. Por tanto, cuadra perfectamente que el caso ordenado sea el más rápido (no hace intercambios), el caso inverso sea el más lento (hace el máximo de intercambios) y el caso aleatorio se quede en un tiempo intermedio.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de Selección:

n	*t ordenado*	*t inverso*	*t aleatorio*
10000	401	350	394
20000	1563	1418	1544
40000	6239	5634	6162
80000	24807	22575	24683
160000	100064	86102	99005

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.
Sí, los datos obtenidos concuerdan con lo esperado ya que la complejidad del algoritmo de Selección es $O(n^2)$. Al duplicar el tamaño del vector de entrada (), el número de operaciones debe multiplicarse por $2^2 = 4$. Por tanto, cuadra, ya que al dividir el tiempo de un valor de entre el anterior, el resultado está siempre en torno a 4.

En cuanto a los diferentes tiempos para un mismo valor de, también concuerdan con lo esperado. El algoritmo de Selección siempre realiza el mismo número de comparaciones sin importar cómo estén colocados los datos inicialmente. Por tanto, cuadra perfectamente que los tiempos en las columnas de ordenado, inverso y aleatorio sean prácticamente idénticos.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de Inserción:

n	*t ordenado*	*t inverso*	*t aleatorio*
10000	0	682	338
20000	0	2714	1370
40000	0	10869	5436
80000	1	43354	21683
160000	3	174148	86085

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.
Sí, los datos obtenidos concuerdan con lo esperado ya que la complejidad del algoritmo de Inserción en sus casos medio y peor es $O(n^2)$. Al duplicar el tamaño del vector de entrada (), el número de operaciones debe multiplicarse por $2^2 = 4$. Por tanto, cuadra, ya que al dividir los tiempos de las columnas "inverso" y "aleatorio" de un valor de entre el anterior, el resultado está siempre en torno a 4.

En cuanto a los diferentes tiempos para un mismo valor de O(n), también concuerdan con lo esperado, ya que la Inserción es extremadamente sensible al orden inicial. Por tanto, cuadra perfectamente que el caso ordenado registre tiempos casi nulos (baja su complejidad a al no tener que hacer ningún desplazamiento), el caso inverso sea el más lento (es el peor caso y hace todos los desplazamientos posibles) y el aleatorio se quede en un tiempo intermedio, tardando aproximadamente la mitad que el inverso.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de QuickSort:

n	*t ordenado*	*t inverso*	*t aleatorio*
10000	3	3	3
20000	5	6	8
40000	11	13	17
80000	23	25	36
160000	48	53	82

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.
Sí, los datos obtenidos concuerdan con lo esperado ya que la complejidad teórica del método Rápido (caso medio y mejor) es O(n log n). Al duplicar el tamaño del vector de entrada (2n), el tiempo de ejecución no se cuadruplica, sino que crece multiplicándose por un factor ligeramente superior a 2. Por tanto, cuadra, ya que al dividir el tiempo de un valor de entre el anterior en la tabla, el resultado se sitúa de forma constante en torno a 2.1 o 2.2, confirmando su eficiencia logarítmica.

En cuanto a los diferentes tiempos para un mismo valor de n, también concuerdan con la implementación del algoritmo. Al escoger el pivote en la posición central del vector (como hace el código de la práctica), se evita caer en el peor caso O(n^2). Por tanto, cuadra perfectamente que los tiempos sean bajísimos en las tres columnas y que el caso aleatorio sea ligeramente más lento que el ordenado o inverso, ya que los vectores desordenados obligan al algoritmo a realizar un mayor número de intercambios (swaps) en la memoria.

TABLA 5= RÁPIDO vs. INSERCIÓN PARA TIEMPOS BAJOS
(tiempos en milisegundos y SIN_OPTIMIZACIÓN y para caso ALEATORIO)

n	*t rapido*	*t insercion*	*t rapido/t insercion*
10	127	149	0,852348993
15	211	239	0,882845188
20	287	307	0,93485342
25	420	399	1,052631579
30	503	517	0,972920696
35	625	586	1,066552901
40	695	705	0,985815603
45	793	845	0,938461538
50	926	962	0,962577963
55	1025	1175	0,872340426
60	1157	1291	0,896204493
65	1271	1312	0,96875
70	1393	1461	0,953456537
75	1534	1567	0,978940651
80	1716	1747	0,982255295
85	1800	1796	1,002227171
90	1941	1932	1,004658385
95	2035	2036	0,999508841
100	2210	2202	1,003633061

Con la tabla solicitada no se obtiene un resultado muy claro, pero para poder responder a la pregunta se hacen pruebas con un mayor

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041\p2>java -Xint p2.InsercionTiemposBajos aleatorio 100000
n=200   Tiempo=4974     Repeticiones=100000
n=205   Tiempo=5136     Repeticiones=100000
n=210   Tiempo=5161     Repeticiones=100000
n=215   Tiempo=5370     Repeticiones=100000
n=220   Tiempo=5439     Repeticiones=100000
n=225   Tiempo=5575     Repeticiones=100000
n=230   Tiempo=5764     Repeticiones=100000
n=235   Tiempo=5895     Repeticiones=100000
n=240   Tiempo=6108     Repeticiones=100000
n=245   Tiempo=6195     Repeticiones=100000
n=250   Tiempo=6444     Repeticiones=100000
n=255   Tiempo=6483     Repeticiones=100000
n=260   Tiempo=6747     Repeticiones=100000
n=265   Tiempo=7015     Repeticiones=100000
n=270   Tiempo=7195     Repeticiones=100000
n=275   Tiempo=7086     Repeticiones=100000
n=280   Tiempo=7253     Repeticiones=100000
n=285   Tiempo=7425     Repeticiones=100000
```

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041\p2>java -Xint p2.RapidoTiemposBajos aleatorio 100000
n=200   Tiempo=4924     Repeticiones=100000
n=205   Tiempo=5039     Repeticiones=100000
n=210   Tiempo=5195     Repeticiones=100000
n=215   Tiempo=5340     Repeticiones=100000
n=220   Tiempo=5512     Repeticiones=100000
n=225   Tiempo=5595     Repeticiones=100000
n=230   Tiempo=5887     Repeticiones=100000
n=235   Tiempo=6007     Repeticiones=100000
n=240   Tiempo=6119     Repeticiones=100000
n=245   Tiempo=6256     Repeticiones=100000
n=250   Tiempo=6509     Repeticiones=100000
n=255   Tiempo=6929     Repeticiones=100000
n=260   Tiempo=7183     Repeticiones=100000
n=265   Tiempo=7280     Repeticiones=100000
```

Tras hacer estas pruebas ves que con valores tan bajos la diferencia es mínima y aproximando, el valor en el que puede notarse la diferencia es entre n = 250 y n = 300.

De cara a implementar el algoritmo opcional para tratar de mejorar los tiempos se estableció un umbral a partir del cual el método empieza a utilizar el algoritmo de inserción. Teóricamente debería mejorar el tiempo, pero, quizás por las características del ordenador o por algún error a la hora de la implementación el tiempo no mejora.

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041\p2>java -Xint p2.RapidoTiempos2 aleatorio 1
n=10000 Tiempo=230      Repeticiones=1
n=20000 Tiempo=438      Repeticiones=1
n=40000 Tiempo=1860     Repeticiones=1
n=80000 Tiempo=6714     Repeticiones=1
n=160000        Tiempo=40238    Repeticiones=1
```

TAREA LAB-3.1

SE LE PIDE:

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Sustracción 1 (O(n)) y Sustracción 2 (O(n^2)): Los tiempos reflejan sus complejidades lineal y cuadrática (multiplicándose por 2 y por 4 respectivamente al duplicar n). Sin embargo, la naturaleza de la recursión por sustracción hace que la profundidad de la pila de llamadas sea igual a n. Esto provoca que ambas clases aborten prematuramente (StackOverflow) para valores pequeños (en torno a n = 8000), impidiendo procesar entradas grandes.

Sustracción 3 (O(2^n)): Los tiempos confirman su naturaleza exponencial. El algoritmo se vuelve intratable casi de inmediato; con solo sumar 1 o 2 unidades al valor de n, el tiempo se multiplica por 2 de forma agresiva, haciendo imposible superar valores de n = 40 o n = 50 en tiempos de ejecución razonables.

Contestar:
¿para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2? ¿por qué sucede eso?
Las clases Sustracción 1 y Sustracción 2 dejan de dar tiempos (abortan) normalmente cuando supera valores justo después de 8000.

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041\p3>java -Xint p3.Sustraccion2 1
 n=1000**TIEMPO=12**nVeces=1
 n=2000**TIEMPO=30**nVeces=1
 n=4000**TIEMPO=126**nVeces=1
 n=8000**TIEMPO=488**nVeces=1
Exception in thread "main" java.lang.StackOverflowError
        at p3.Sustraccion2.rec2(Sustraccion2.java:10)
        at p3.Sustraccion2.rec2(Sustraccion2.java:15)
        at p3.Sustraccion2.rec2(Sustraccion2.java:15)
```

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasUO306041\p3>java -Xint p3.Sustraccion1 1
 n=1000**TIEMPO=0**nVeces=1
 n=2000**TIEMPO=2**nVeces=1
 n=4000**TIEMPO=0**nVeces=1
 n=8000**TIEMPO=0**nVeces=1
Exception in thread "main" java.lang.StackOverflowError
        at p3.Sustraccion1.rec1(Sustraccion1.java:9)
        at p3.Sustraccion1.rec1(Sustraccion1.java:14)
        at p3.Sustraccion1.rec1(Sustraccion1.java:14)
```

Esto ocurre porque se desborda la pila y se lanza StackOverflowError. Al ser algoritmos recursivos de sustracción simple (n-1), cada llamada genera un nuevo marco en la pila de ejecución.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80? Razonar la respuesta.
El algoritmo en Sustracción 3 tiene una complejidad de $O(n^2)$ debido a que realiza dos llamadas recursivas por cada unidad que se resta a n.

- Razonamiento: El número total de llamadas es aproximadamente 2^n. Para n = 80, tenemos 2^{80} operaciones.
- Cálculo:
    1. Operaciones: $1{,}2089 * 10^{24}$.
    2. En años serian aproximadamente 38.000.000 de años
- Conclusión: Es un algoritmo intratable para valores de superiores a 40-50.

Implementar una clase Sustracción4.java con una complejidad O(n3) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=100, 200, 400, 800, … (hasta FdT).

n	*Sust4 t(ms)*
100	254
200	510
300	773
400	1029
500	1279
600	1552
700	1801
800	fdt
900	fdt

Implementar una clase Sustracción5.java con una complejidad O(3n/2) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, … (hasta FdT).

n	*Sust5 t(ms)*
30	2246
32	2402
34	2554
36	2704
38	2873
40	2997
42	fdt
44	fdt
46	fdt
48	fdt
50	fdt

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80? Razonar la respuesta.
En este caso la complejidad es de $O(3^{n/2})$, por tanto:

1. Con n = 80, $3^{80/2} = 3^{40}$ que es más o menos $1{,}2157 * 10^{19}$
2. Por tanto, en años seria más o menos 385,5 años

SE LE PIDE:

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.
División 1 (O(n)) y División 3 (O(n)): Ambas clases demuestran un comportamiento puramente lineal. Al duplicar el tamaño de la entrada (2n), el tiempo de ejecución empírico se multiplica por un factor muy cercano a 2. Al ser recursividad por división, la profundidad de la pila es logarítmica, lo que explica por qué estas clases no abortan y son capaces de procesar valores gigantescos (hasta n = 1.024.000.000) en fracciones de segundo.

División 2 (O(n log n)): Los tiempos observados concuerdan con la complejidad. Al duplicar el tamaño del problema, el tiempo no se cuadruplica (como pasaría en un $O(n^2)$), sino que crece multiplicándose por un factor ligeramente superior a 2 (aprox. 2.1 o 2.2). Esto permite procesar volúmenes masivos de datos manteniendo una eficiencia altísima, de nuevo, sin desbordar la pila del sistema gracias a la división del problema.

Implementar una clase Division4.java con una complejidad O(n2) (con a<bk) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, … (hasta FdT).

n	*Div4 t(ms)*
1000	14
2000	42
4000	169
8000	643
16000	2593
32000	10343
64000	41095

Implementar una clase Division5.java con una complejidad O(n2) (con a>bk) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, … (hasta FdT).

n	*Div5 t(ms)*
1000	31
2000	88
4000	345
8000	1344
16000	5403
32000	21794

SE LE PIDE:

Tras analizar la complejidad de los diversos algoritmos que hay dentro de las dos clases, ejecutarlas y tras poner en una tabla los tiempos obtenidos, comparar la eficiencia de cada algoritmo.

Para VectorSuma.java:

Método	*Algoritmo*	*t*	*Complejidad*
Método 1 (**sum1)**	Iterativo	532ns	O(n)
Método 2 (**sum2)**	Recursivo (Sustracción)	1211ns	O(n)
Método 3 (**sum3)**	Recursivo (División)	2852ns	O(n)

La recursividad (Métodos 2 y 3) introduce penalizaciones graves de tiempo y recursos debido a la gestión de la pila de llamadas. Para un problema elemental como sumar o recorrer todos los elementos de un array, la solución iterativa siempre será la más eficiente en la práctica. El diseño Divide y Vencerás (Método 3) solo justifica su sobrecoste temporal cuando consigue bajar la complejidad teórica del problema (por ejemplo, pasando de un $O(n^2)$ a un $O(n \log n)$. Por tanto, en problemas a gran escala si estaría justificado el uso de DyV.

Para Fibonacci.java

Método	*Algoritmo*	*Tiempo*	*Complejidad*
Método 1	Iterativo	374 ns	O(n)
Método 2	Iterativo alternativo	539 ns	O(n)
Método 3	Ecuación matricial	462 ns	O(n) u O(log n)
Método 4	Recursivo puro (DyV)	3.335 ms	O(2^n)

PRIMERA PARTE

Resolver el problema mediante un algoritmo trivial con una clase p3p.PuntosTrivial.java, que calcule la solución calculando la distancia entre todo par de puntos diferentes, para así saber al final el par de puntos buscado y su distancia mínima.

Ejecutar la clase metiendo como parámetro cada uno de los nombres de los ficheros aportados, para comprobar el buen funcionamiento del algoritmo implementado:

n	*tTrivial*
1000	23
2000	95
4000	337
8000	1340
16000	5279
32000	21115
64000	77690

SEGUNDA PARTE

Resolver el problema mediante un algoritmo trivial con una clase p3p.PuntosDyV.java, que calcule la solución calculando la distancia entre todo par de puntos diferentes, para así saber al final el par de puntos buscado y su distancia mínima.

Ejecutar la clase metiendo como parámetro cada uno de los nombres de los ficheros aportados, para comprobar el buen funcionamiento del algoritmo implementado:

n	*tDyV*
1000	1
2000	1
4000	4
8000	6
16000	14
32000	28
64000	56